

Testing Analysis

Boundary Value Testing

For Boundary Value Testing, the `applyDiscount(double percentage)` method from the `Item` class was selected. This method applies a discount to the selling price of an item based on a numerical percentage value and includes validation logic that restricts the accepted input range. The valid domain for the percentage parameter is from 0 to 100 inclusive. Any value outside this range results in an `IllegalArgumentException`, making this method suitable for boundary-focused testing. Boundary Value Testing was conducted by evaluating values at and around the limits of the valid input range. The tested values included one value below the minimum boundary (-1), the minimum value (0), one value just above the minimum (1), one value just below the maximum (99), the maximum value (100), and one value above the maximum boundary (101). When the percentage was set to -1 or 101, the method correctly rejected the input by throwing an exception. When valid boundary values were provided, the selling price was updated correctly. A discount of 0 left the selling price unchanged, while a discount of 100 reduced the selling price to zero. These results confirm that the method behaves correctly at boundary limits and properly handles invalid values.

Boundary Case	Percentage Value	Description	Result
Max +1	101	Above the valid range of discount	Exception thrown – Passing the range
Max	100	Full discount	Selling price becomes zero
Max -1	99	Almost having a full discount	Selling price is very close to zero
Min +1	1	Very small discount that might be valid	Selling price changed just a little
Min	0	No discount	Selling price unchanged
Min -1	-1	Below valid range	Exception thrown – Do not pass the range

Equivalence Class Testing

For Equivalence Class Testing, the `increaseStock(int quantity)` method was chosen. This method increases the quantity of an item while ensuring that only positive numerical values are accepted. The input domain was divided into equivalence classes based on the expected behavior of the method. All positive integer values form a valid equivalence class, as they result in a successful increase of stock. Zero and negative values form invalid equivalence classes, as they are rejected by the method. Representative values from each equivalence class were tested to verify correct behavior. When a positive quantity was provided, the stock level increased as expected and the item's

availability status was updated accordingly. When zero or negative values were used as input, the method consistently threw an exception, preventing invalid stock updates. This demonstrates that the method correctly distinguishes between valid and invalid input classes and applies the same behavior to all values within each class.

Condition	Example	Expected output	Valid/Invalid
quantity > 0	7	Increase of stock , gives the store an updated stock	Valid
quantity = 0	0	Exception thrown , it doesnt make any change in the stock	Invalid
quantity < 0	-7	Exception thrown , we can't have a negative quatity to update	Invalid

Code Coverage Testing

For Code Coverage Testing, the decreaseStock(int quantity) method was selected because it contains conditional logic that controls the execution flow of the method. This method validates whether the given quantity is greater than zero and whether it does not exceed the available stock. Depending on the evaluation of these conditions, the method either updates the item quantity and the number of sold items or throws an exception, making it suitable for coverage analysis.

Statement Coverage: Statement coverage was achieved by executing every executable statement in the method at least once. This was done by using a valid test case where the stock was successfully decreased and invalid test cases where an exception was thrown. As a result, all statements inside the method were executed, achieving 100% statement coverage.

Branch Coverage: Branch coverage was achieved by testing both possible outcomes of the conditional logic. The test suite included cases where the condition evaluated to false, leading to normal execution, and cases where the condition evaluated to true, leading to an exception. This ensured that all branches of the decision structure were executed, resulting in 100% branch coverage.

Condition Coverage: Condition coverage was satisfied by designing test cases that caused each individual condition within the logical expression to evaluate to both true and false. The tests included cases where the quantity was less than or equal to zero and cases where the quantity was greater than the available stock, as well as valid cases where both conditions were false. This ensured that all conditions were fully evaluated, achieving 100% condition coverage.

MC/DC Coverage: Modified Condition/Decision Coverage (MC/DC) was achieved by demonstrating that each condition independently affects the final decision outcome. This was accomplished by changing one condition at a time while keeping the other condition unchanged and observing a change in the method's behavior. Separate test

cases were used to show that each condition could independently cause the method to throw an exception. Therefore, 100% MC/DC coverage was achieved.